

In the name of ALLAH



**Amirkabir University of Technology
(Tehran Polytechnic)**

**Deep Neural Network Course By Dr. Safabakhsh
spring 2025**

Assignment (05)

Mohammad Sadegh Eftekhari

403131014

Sadegh.eft@aut.ac.ir

Contents

IMPORTS	2
Before we begin	3
Q1: Preprocessing.....	3
Vocabulary Construction.....	3
Tokens to Input	4
Splitting the dataset.....	4
Creating DataLoaders.....	4
Q2: A Simple RNN Model	4
'Train' & 'Validation' loop.....	5
Prediction (Test set Evaluation).....	5
Visualizer	6
Training the model and Visualization.....	6
Q3: Replace the Recurrent Layer with an LSTM.....	7
Q4: Hyperparameter Sensitivity	9
making a new data loader using the best 'seq_len' parameter value.....	10
Q5: Bi-Directional LSTM	10
Q6: Hybrid CNN-LSTM Model	12
Q7: Error Analysis.....	15
Analyzing the results	15
True Positive	15
True Negatives.....	16
False Positive	16
False Negative	16

IMPORTS

1. `torch + torch.nn`
For building and training the neural network model (embedding, RNN, Linear, etc.).
2. `torch.utils.data`
To build custom Dataset and DataLoader classes for training/validation/test batching.
3. `hazm`
For Persian NLP tasks like normalization and tokenization.
4. `collections.Counter`
To count token frequencies for building the vocabulary.
5. `tqdm.auto`
For progress bars during iterative processes like model training and evaluation.
6. `warnings`
For suppressing warning messages that might appear during code execution.
7. `timeit.default_timer`
For accurately measuring the execution time of the training process.
8. `pandas`
To load and handle tabular data (in this case, the Persian spam dataset).
9. `String`
To remove punctuation marks during preprocessing.
10. `matplotlib.pyplot`
For the primary plotting capabilities used to visualize training and evaluation loss curves.
11. `sklearn.metrics`
standard functions (`accuracy_score`, `precision_score`, `recall_score`, `f1_score`) to evaluate the classification performance of the trained models.
12. `sklearn.model_selection.train_test_split`
To properly split the dataset into train, validation, and test sets.
13. `textwrap`
Used to neatly wrap and indent long text lines when printing sample emails during error analysis.

Before we begin

Before getting into the main parts of the code, we have to set some things up.

The 1st thing is to filter any warnings right out of the gate, we do this by calling the method from the 'warnings' library and passing the value of 'ignore'. We then want to setup our device, if the 'CUDA' is available we use that, if not we just use our 'CPU'. The last thing is to setup a timer function which is going to calculate the time and print it for us.

Q1: Preprocessing

The link to the data was provided by the TA in the assignment instructions, it was a link to the 'kaggle' website, so I just downloaded it and loaded it into the code using the 'pd.read_csv'. The dataset we're using here is a Persian spam email dataset, containing labeled emails as either "ham" (normal) or "spam".

We start by applying a preprocessing pipeline using the Hazm library:

1. Normalize the text (correct_spacing, remove_diacritics, remove_specials_chars, ect.)
2. Tokenize using "hazm.word_tokenize".
3. Remove all punctuation (including Persian-specific ones like 'and ').
4. Remove stopwords.
5. Join tokens back into a clean string version (this helps in manual inspection later).

Vocabulary Construction

After tokenizing and cleaning the text, we need to convert our words into numbers, but before we can do that, we have to build a vocabulary that maps each word (token) to a unique integer.

To build this vocabulary:

- We flatten all token lists from all emails into one giant list of tokens.
- We use Counter to count how often each token appeared.
- Then we sort the tokens (just for consistency) and build the vocab

We also add 2 special tokens:

1. **<PAD>**: This token is used to fill out the end of sequences that are shorter than the max sequence length. We want every input to the model to be of the same length, and instead of using 0 or a random value, we reserve a proper token for this.
2. **<UNK>**: This stands for "unknown." It's basically a fallback token for when we see a word at test time that we didn't see during training.

From this, we create "token2idx" which maps tokens to indices, which allows us to easily convert our text into numerical format before passing it to the neural network.

Tokens to Input

Now we define a function that takes a list of tokens and turns them into a fixed-length list of integers

- We use `.get()` so if a token doesn't exist in `token2idx`, it gets mapped to `<UNK>`.
- We pad with `<PAD>` if it's shorter than `max_len` (which we set to 50).
- We cut it short if it's longer.

Splitting the dataset

To train and evaluate the model properly, we split the dataset into three parts:

- 70% training
- 10% validation
- 20% test

We use `train_test_split()` twice, first to split train from the rest, and then to split the rest into validation and test sets. I also set `stratify=label` in the splits to make sure that the distribution of ham vs. spam stays balanced in all sets.

Creating DataLoaders

We define a custom “EmailDataset” class that subclasses “`torch.utils.data.Dataset`” to convert DataFrame rows into (input_tensor, label_tensor) pairs in a clean and reusable way that works with DataLoader. This class returns (x, y) pairs:

- x is the padded tensor of input IDs
- y is the binary label

Then we create DataLoader instances for train, validation, and test sets. And we use a batch size of 64, and `shuffle=True` for the training loader so the batches are randomized each epoch.

Q2: A Simple RNN Model

Now we define our base model — a simple RNN classifier. Here's what it includes:

1. **Embedding Layer:** Takes the word indices and turns them into vectors of dimension “`embed_dim`”. This layer is trainable, meaning the embedding vectors are learned during training.
2. **Recurrent Layer (RNN):** We use `nn.RNN` with hidden size “`hidden_dim`”.
3. **Fully Connected Layer:** After the RNN processes the input, we take the last hidden state and pass it through a linear layer to output a single value. This value represents the probability that the input email is spam.

'Train' & 'Validation' loop

Like any other Neural Network task, we also need a training and validation (evaluation) loop, this one is not any different. To define a training and validation loop, we obviously need some parameters, the ones common in any other loop, are the 'model', the 'loss' and 'optimizer', the data 'loaders' which in this case would be the train and validation data loaders, the 'device' and finally the 'number of epochs', in our case, we also have an extra parameter, the 'show_result' which if set to 'True' it prints the results of each epoch, I defined this parameter because when we're doing grid search, it prints too many results and it's confusing. We then use the 'timit' library to track the 'run time'.

We already know that we need to monitor the training and validation losses, so we know that we need a method of storing the losses, so we define lists for this purpose, we also do the same for the 'accuracies'.

We now define a loop which is going to be repeated for the number of epochs, this loop is going to be the same loop that is being used for the validation set, so what we're doing is, we train the model for one epoch and then validate the model on the validation set, we then do the same thing for the number of epochs we chose.

We set the model to the train mode, we then define another loop which is going to go through our batches, for each batch we perform the following tasks:

1. Move the data to the 'device'
2. Clear the previous gradient
3. Compute the model prediction
4. Use the loss function to calculate the loss
5. Perform backpropagation
6. Update the model weights
7. Calculate the average loss

We then predict the label (anything above "0.5" is predicted as 'spam'). Then if the 'show_result' is 'True' we present the results. We then move on to evaluation steps. we first, set the model to evaluation mode, disable gradient computation for evaluation, and iterate over the batches.

1. Move the data to the 'device'
2. Compute the model prediction (test data)
3. Compute the loss of the batch
4. Store the number for 'total' and 'correct' predictions (for accuracy)
5. Calculate the average loss and accuracy

After the main loop (for the number of epochs) is done, we then use the functions of the 'sklearn.metrics' ('precision', 'recall', 'f1') to calculate these metrics and return them (as the assignment has stated should be done). We then use the "print_train_time" function to calculate the run time and print it. after this, we repeat what we did for the training loop, which is checking the value of the 'show_result' and printing the results if 'True'.

Prediction (Test set Evaluation)

This method is used to do almost what we did for the ‘validation’ set, but on the ‘test’ set and without tracking the loss. So, we:

1. Move the data to the ‘device’
 2. Predict the output
 3. Store the number for ‘total’ and ‘correct’ predictions (for accuracy)
- We then compute the accuracy, print and then return it (for compare table).

Visualizer

For visualization function, we don’t really do anything special, we just take-in the losses and accuracies, we then define 2 subplots, and plot losses on one, and the accuracies on the other.

Training the model and Visualization

All we need to do for this section is just calling the define the model, loss function and the optimizer, and then call the ‘train and validation loop’ function, and the ‘visualizer’ after that. For the loss function we’re using the “BCEWithLogitsLoss” and for the optimizer we’re using the “Adam” optimizer.

This is what running the ‘training and validation loop’ output looks like:

```
Epoch 1/10 - Train Loss: 0.6920, Accuracy: 58.29%
Epoch 1/10 - Val Loss: 0.6800, Accuracy: 61.00%

Epoch 2/10 - Train Loss: 0.6473, Accuracy: 69.14%
Epoch 2/10 - Val Loss: 0.6606, Accuracy: 65.00%

Epoch 3/10 - Train Loss: 0.5947, Accuracy: 74.43%
Epoch 3/10 - Val Loss: 0.6161, Accuracy: 67.00%

Epoch 4/10 - Train Loss: 0.5136, Accuracy: 77.29%
Epoch 4/10 - Val Loss: 0.5429, Accuracy: 77.00%

Epoch 5/10 - Train Loss: 0.3964, Accuracy: 85.86%
Epoch 5/10 - Val Loss: 0.4892, Accuracy: 78.00%

Epoch 6/10 - Train Loss: 0.3074, Accuracy: 90.29%
Epoch 6/10 - Val Loss: 0.4819, Accuracy: 79.00%

Epoch 7/10 - Train Loss: 0.2391, Accuracy: 93.43%
Epoch 7/10 - Val Loss: 0.3970, Accuracy: 81.00%

Epoch 8/10 - Train Loss: 0.2011, Accuracy: 95.00%
Epoch 8/10 - Val Loss: 0.3769, Accuracy: 86.00%

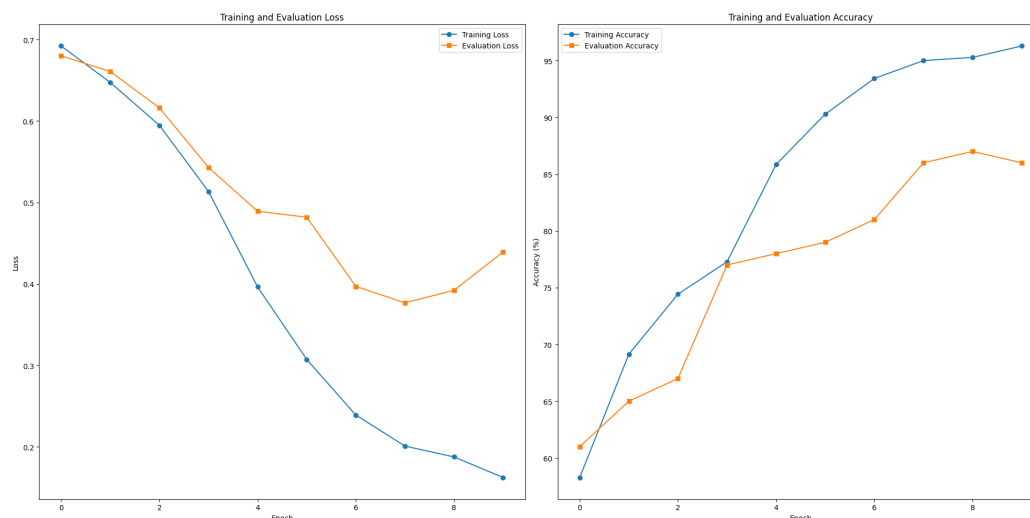
Epoch 9/10 - Train Loss: 0.1877, Accuracy: 95.29%
Epoch 9/10 - Val Loss: 0.3922, Accuracy: 87.00%

Epoch 10/10 - Train Loss: 0.1625, Accuracy: 96.29%
Epoch 10/10 - Val Loss: 0.4394, Accuracy: 86.00%

Val Precision: 0.7903, Recall: 0.9800, F1-Score: 0.8750

Train time on cuda: 0.065 minutes
```

And now the output of the ‘visualizer’ function:



The outputs show that we get the best results at the 8th and 9th epochs, and on the 10th the validation accuracy starts to increase, meaning the model is now overfitting. So I'm going to use '9' as the number of epochs from now on.

Q3: Replace the Recurrent Layer with an LSTM

In the previous section, we used a basic RNN for our classifier. It worked, but regular RNNs are known to struggle with long sequences because they can't remember important information from earlier steps very well. That's exactly why we replace the RNN layer with a Long Short-Term Memory (LSTM) layer in this section.

The difference between RNNs and LSTM is that RNNs have trouble learning long-term dependencies, they tend to forget things quickly. LSTMs were specifically designed to solve this. They have extra gates (input, forget, output) that help them remember or forget information as needed.

The new model is very similar to the previous one, we're just replacing the `nn.RNN` layer with "`nn.LSTM`". Here's a breakdown of the layers:

Embedding Layer: Same as before — maps token indices into dense vectors of dimension 100.

LSTM Layer: Takes in the embedded sequence and outputs two things:

- output: the full sequence of hidden states (we don't use this here).
- hidden: the final hidden state and the cell state.

We only care about the final hidden state, so we take `hidden[0]`.

Dropout Layer: We add a dropout layer right before the final linear layer, but this is just for the next task, so I don't have to write another model just for that. So I just set the value for the 'dropout' layer to '0' which means its effectively just not doing anything.

Fully Connected Layer: Just like before, this takes the hidden state and maps it to a single output value (spam or ham).

We then just do the regular task of defining the model, loss function and the optimizer and then calling the functions for training and visualization, and these are the results:

```
Epoch 1/9 - Train Loss: 0.6935, Accuracy: 52.43%
Epoch 1/9 - Val Loss: 0.6848, Accuracy: 61.00%

Epoch 2/9 - Train Loss: 0.6687, Accuracy: 68.00%
Epoch 2/9 - Val Loss: 0.6731, Accuracy: 61.00%

Epoch 3/9 - Train Loss: 0.6372, Accuracy: 77.29%
Epoch 3/9 - Val Loss: 0.6483, Accuracy: 67.00%

Epoch 4/9 - Train Loss: 0.5713, Accuracy: 84.00%
Epoch 4/9 - Val Loss: 0.5609, Accuracy: 72.00%

Epoch 5/9 - Train Loss: 0.4180, Accuracy: 89.00%
Epoch 5/9 - Val Loss: 0.4003, Accuracy: 83.00%

Epoch 6/9 - Train Loss: 0.2589, Accuracy: 93.86%
Epoch 6/9 - Val Loss: 0.3442, Accuracy: 87.00%

Epoch 7/9 - Train Loss: 0.1751, Accuracy: 97.00%
Epoch 7/9 - Val Loss: 0.2782, Accuracy: 89.00%

Epoch 8/9 - Train Loss: 0.1331, Accuracy: 97.86%
Epoch 8/9 - Val Loss: 0.2457, Accuracy: 92.00%

Epoch 9/9 - Train Loss: 0.0810, Accuracy: 98.14%
Epoch 9/9 - Val Loss: 0.2937, Accuracy: 92.00%

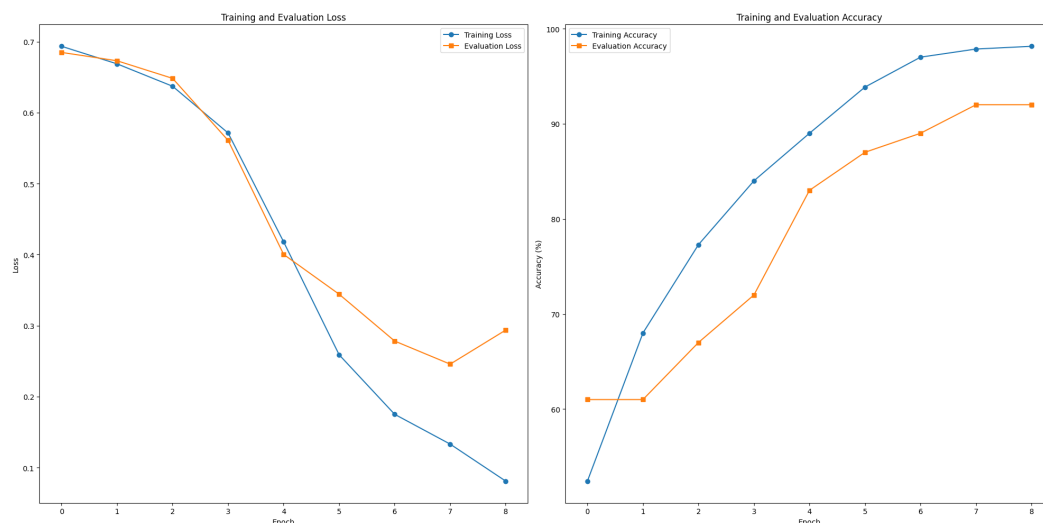
Val Precision: 0.9038, Recall: 0.9400, F1-Score: 0.9216

Train time on cuda: 0.043 minutes

Final Test Evaluation: 100% |██████████| 4/4 [00:00<00:00, 85.34it/s]

Test Accuracy: 84.00%
Precision: 0.8542, Recall: 0.8200, F1: 0.8367
```

And now the output of the 'visualizer' function:



Looks like reducing the number of epochs from '10' to '9' did prevent overfitting after all. As can be seen the model definitely out-performed the last model.

Both models demonstrate learning, as evidenced by decreasing training loss and increasing training accuracy. However, Model 2 (the LSTM model) significantly outperformed Model 1. Model 2 achieved a higher peak evaluation accuracy, and a lower minimum evaluation loss.

The superior performance of Model 2 (the LSTM model) likely stems from its architectural design, which is inherently better suited for capturing complex patterns, especially if the data possesses sequential characteristics or long-range dependencies. This capability enables them to learn more nuanced and temporally distant relationships within the data that a simpler ANN architecture (like that used in Model 1) might struggle to capture.

Q4: Hyperparameter Sensitivity

Now that we have our LSTM model up and running, we want to see how sensitive it is to different parameters. This part of the assignment asks us to vary several key hyperparameters and observe how they affect the model's performance.

We tested the model with different values for the following 4 parameters:

- Sequence lengths (max_len) = [30, 50, 100]
- embedding dimensions = [50, 100, 200]
- hidden layer dimensions = [32, 64, 128]
- dropout rates = [0.0, 0.3, 0.5]

We used nested loops to try every combination of the parameters above, so a total of '81'.

For each combination, we:

1. Regenerated the token sequences using the selected seq_len.
2. Re-split the dataset into train/val/test to stay consistent.
3. Created a new model with the selected embed_dim, hidden_dim, and dropout.
4. Trained the model for 9 epochs.
5. Stored the maximum validation accuracy reached during training.

All the results were saved in a list and then converted into a pandas DataFrame for easy inspection and filtering.

I then stored the best values for each parameter in a series dataframe to use the future models.

Displaying '81' rows wouldn't be logical, so I'm just going to present the best values for our parameters and also the accuracy for that row.

```
Best Parameters:
seq_len      50.0
embed_dim    200.0
hidden_dim    128.0
dropout_rate  0.3
val_accuracy  97.0
```

Based on the provided table, the model appears most sensitive to changes in `hidden_dim` and `dropout_rate`.

hidden_dim: Increasing the “hidden_dim” often resulted in the most substantial gains in accuracy. For instance, with “seq_len=50”, “embed_dim=200”, “dropout_rate=0.3”, increasing “hidden_dim” from 32 to 64 to 128 boosted accuracy from 84% to 92% and finally to 97%. This indicates that model capacity, as controlled by “hidden_dim”, is a critical factor.

dropout_rate: The model is also very sensitive to the “dropout_rate”, but its effect is more about finding the right balance. While the optimal dropout (0.3) improved the best configuration, an inappropriate dropout rate could significantly degrade performance. For example, increasing dropout from 0.0 to 0.5 caused accuracy to plummet from 83% to 68%. This highlights that incorrect regularization can be very detrimental.

While “embed_dim” also shows a clear positive impact on accuracy, the largest individual swings in performance often correlate with changes in “hidden_dim” or the application of an inappropriate “dropout_rate”.

making a new data loader using the best 'seq_len' parameter value

After running the full hyperparameter sweep, we found the best-performing setup, and now we want to use it going forward.

But here's the thing: one of the parameters we changed during tuning was `seq_len`, and since that affects how we generate our input sequences, we need to regenerate the input IDs using this best `seq_len` value.

Q5: Bi-Directional LSTM

So far, all the RNNs and LSTMs we've used have been unidirectional, meaning they only process the input from left to right (start to end). But when it comes to language, sometimes the context after a word is just as important as the context before it. That's exactly what Bidirectional LSTMs (BiLSTMs) are designed for.

Instead of a single LSTM layer that reads the input from start to end, a BiLSTM uses two LSTMs:

- One goes forward through the sequence.
- The other goes backward.

At the end, the two hidden states are concatenated, so the final representation includes information from both directions of the input text.

The BiLSTM model is very similar to the one we used before — with just a few key changes:

1. **LSTM Layer:** We pass `bidirectional=True` to `nn.LSTM`. This causes PyTorch to use two separate LSTM layers internally (one forward, one backward).
2. **Output Hidden State:** Since we now have two hidden states (one from each direction), we combine them using `torch.cat()`. This gives us a tensor of shape `[batch_size, hidden_dim * 2]`.
3. **Fully Connected Layer:** The input size of this layer needs to be doubled since the LSTM now outputs `hidden_dim * 2`.

Like every other model we've trained till now, we just define the model, loss function and the optimizer and then just call the train/validation function + test accuracy + loss and accuracy visualization:

```
Epoch 1/9 - Train Loss: 0.6569, Accuracy: 66.57%
Epoch 1/9 - Val Loss: 0.6169, Accuracy: 75.00%

Epoch 2/9 - Train Loss: 0.5638, Accuracy: 88.86%
Epoch 2/9 - Val Loss: 0.5566, Accuracy: 79.00%

Epoch 3/9 - Train Loss: 0.4706, Accuracy: 94.43%
Epoch 3/9 - Val Loss: 0.4840, Accuracy: 84.00%

Epoch 4/9 - Train Loss: 0.3521, Accuracy: 96.86%
Epoch 4/9 - Val Loss: 0.3814, Accuracy: 87.00%

Epoch 5/9 - Train Loss: 0.2146, Accuracy: 97.86%
Epoch 5/9 - Val Loss: 0.2492, Accuracy: 90.00%

Epoch 6/9 - Train Loss: 0.1073, Accuracy: 98.86%
Epoch 6/9 - Val Loss: 0.1779, Accuracy: 93.00%

Epoch 7/9 - Train Loss: 0.0471, Accuracy: 99.00%
Epoch 7/9 - Val Loss: 0.1528, Accuracy: 95.00%

Epoch 8/9 - Train Loss: 0.0192, Accuracy: 99.86%
Epoch 8/9 - Val Loss: 0.1603, Accuracy: 95.00%

Epoch 9/9 - Train Loss: 0.0116, Accuracy: 99.86%
Epoch 9/9 - Val Loss: 0.1636, Accuracy: 95.00%

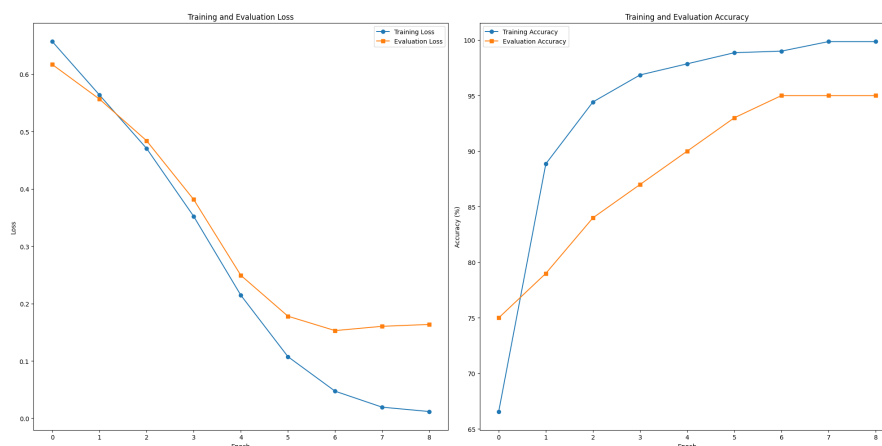
Val Precision: 0.9787, Recall: 0.9200, F1-Score: 0.9485

Train time on cuda: 0.034 minutes

Final Test Evaluation: 100% | 4/4 [00:00:00:00, 127.03it/s]

Test Accuracy: 94.50%
```

And now the output of the 'visualizer' function:



The Bi-directional LSTM achieved a slightly higher peak evaluation accuracy and lower loss compared to the standard LSTM. This improvement is likely because the BiLSTM processes sequences in both forward and backward directions, providing a richer contextual understanding for each token.

However, the BiLSTM also showed signs of overfitting or plateauing in performance earlier, as its evaluation loss began to flatten and accuracy stopped improving. The standard LSTM continued to show modest improvements on the evaluation set a little bit longer. The BiLSTM's increased model capacity (due to processing in two directions and a larger final linear layer) likely allowed it to learn the training data faster but also made it more susceptible to fitting noise and reaching its generalization limit sooner.

Q6: Hybrid CNN-LSTM Model

This task is about designing a model that mixes convolutional layers and recurrent layers — the idea is to let CNNs handle local patterns, and let LSTM capture long-range dependencies over those local patterns.

The layers are as follows:

1. Embedding Layer: Takes token indices and maps them to trainable dense vectors of size `embed_dim`.

2. 1D Convolutional Block: We used a loop to stack multiple CNN layers:

First, we permute the embedding output to shape `[batch, embed_dim, seq_len]` since PyTorch's `Conv1d` expects the channel dimension in the middle.

Each CNN layer does:

- **Convolution:** Detects local patterns across token sequences
- **ReLU:** Non-linearity
- **MaxPool1d:** Reduces sequence length, makes representation more abstract

We do this as many times as because `“num_cnn_layers”`.

3. LSTM Layer: After convolution, we permute again to make the shape `[batch, seq_len, num_filters]`, because LSTM expects time-steps second. LSTM now reads this condensed sequence and learns dependencies between abstracted CNN features, not raw word vectors. This can help it generalize better.

4. Dropout + Fully Connected Layer: Dropout is applied to avoid overfitting. Then we take the final hidden state of the LSTM and pass it to a linear layer to produce the final prediction.

I then define the model and set the value for the parameter `“num_cnn_layers”` to `‘1’`, the rest is like before, loss function, optimizer, train and validate, test, and then visualize.

This is the output of the train/validation loop (train/validation loss and accuracy) + test accuracy:

```

Epoch 1/9 - Train Loss: 0.6701, Accuracy: 62.14%
Epoch 1/9 - Val Loss: 0.6592, Accuracy: 65.00%

Epoch 2/9 - Train Loss: 0.5264, Accuracy: 80.14%
Epoch 2/9 - Val Loss: 0.4754, Accuracy: 84.00%

Epoch 3/9 - Train Loss: 0.2626, Accuracy: 94.71%
Epoch 3/9 - Val Loss: 0.4966, Accuracy: 81.00%

Epoch 4/9 - Train Loss: 0.1923, Accuracy: 95.14%
Epoch 4/9 - Val Loss: 0.2817, Accuracy: 90.00%

Epoch 5/9 - Train Loss: 0.1027, Accuracy: 98.71%
Epoch 5/9 - Val Loss: 0.2636, Accuracy: 89.00%

Epoch 6/9 - Train Loss: 0.0531, Accuracy: 99.14%
Epoch 6/9 - Val Loss: 0.2254, Accuracy: 92.00%

Epoch 7/9 - Train Loss: 0.0297, Accuracy: 99.57%
Epoch 7/9 - Val Loss: 0.2126, Accuracy: 93.00%

Epoch 8/9 - Train Loss: 0.0155, Accuracy: 99.86%
Epoch 8/9 - Val Loss: 0.2012, Accuracy: 93.00%

Epoch 9/9 - Train Loss: 0.0108, Accuracy: 99.86%
Epoch 9/9 - Val Loss: 0.1983, Accuracy: 94.00%

Val Precision: 0.9783, Recall: 0.9000, F1-Score: 0.9375

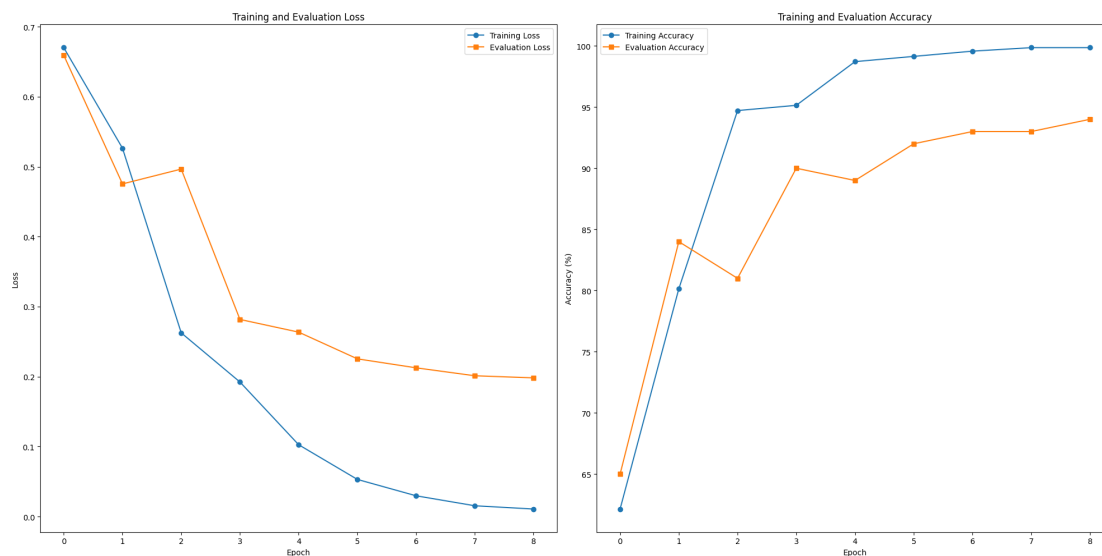
Train time on cuda: 0.038 minutes

Final Test Evaluation: 100%|██████████| 4/4 [00:00<00:00, 116.67it/s]

Test Accuracy: 90.50%
Precision: 0.8932, Recall: 0.9200, F1: 0.9064

```

And now the output of the 'visualizer' function:



Then I tried setting the value of the parameter “num_cnn_layers” to ‘2’ to see if it makes any difference:

```
Epoch 1/9 - Train Loss: 0.6794, Accuracy: 60.29%
Epoch 1/9 - Val Loss: 0.6714, Accuracy: 63.00%

Epoch 2/9 - Train Loss: 0.6078, Accuracy: 76.00%
Epoch 2/9 - Val Loss: 0.5721, Accuracy: 77.00%

Epoch 3/9 - Train Loss: 0.3184, Accuracy: 93.57%
Epoch 3/9 - Val Loss: 0.2399, Accuracy: 92.00%

Epoch 4/9 - Train Loss: 0.0947, Accuracy: 98.14%
Epoch 4/9 - Val Loss: 0.3496, Accuracy: 89.00%

Epoch 5/9 - Train Loss: 0.0406, Accuracy: 99.29%
Epoch 5/9 - Val Loss: 0.3562, Accuracy: 89.00%

Epoch 6/9 - Train Loss: 0.0226, Accuracy: 99.71%
Epoch 6/9 - Val Loss: 0.3050, Accuracy: 92.00%

Epoch 7/9 - Train Loss: 0.0100, Accuracy: 100.00%
Epoch 7/9 - Val Loss: 0.2421, Accuracy: 93.00%

Epoch 8/9 - Train Loss: 0.0073, Accuracy: 100.00%
Epoch 8/9 - Val Loss: 0.1981, Accuracy: 94.00%

Epoch 9/9 - Train Loss: 0.0058, Accuracy: 100.00%
Epoch 9/9 - Val Loss: 0.2788, Accuracy: 91.00%

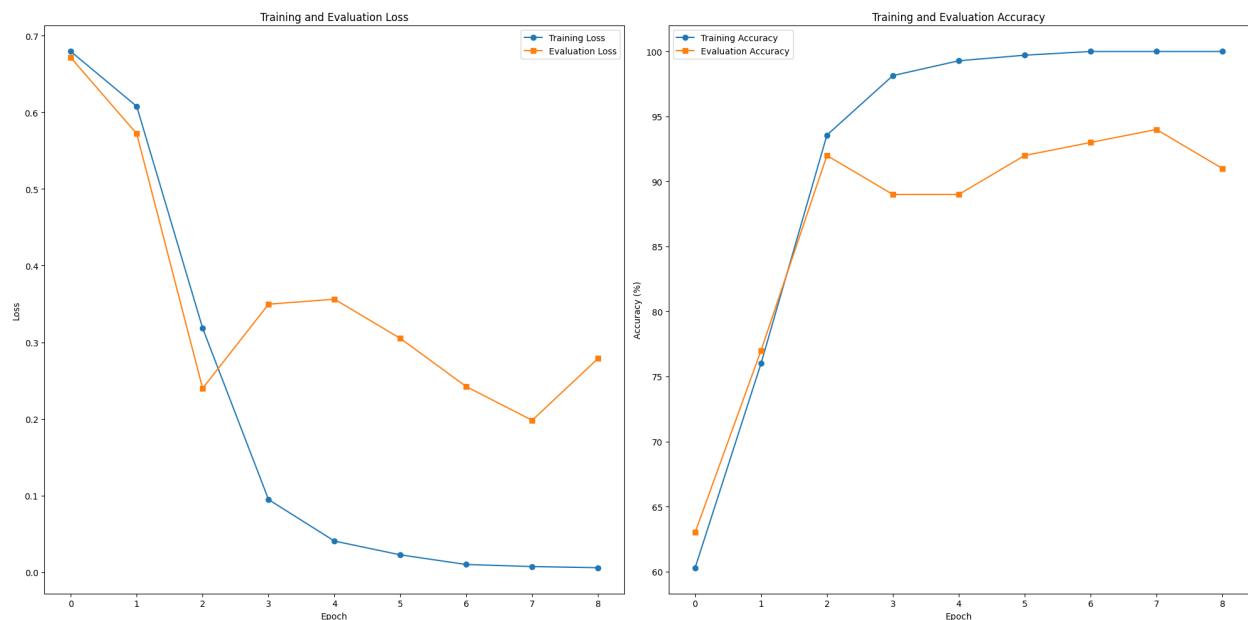
Val Precision: 0.9556, Recall: 0.8600, F1-Score: 0.9053

Train time on cuda: 0.039 minutes

Final Test Evaluation: 100% | 4/4 [00:00<00:00, 117.94it/s]

Test Accuracy: 93.00%
Precision: 0.9479, Recall: 0.9100, F1: 0.9286
```

And now the output of the ‘visualizer’ function:



Seems like it did improve, the test accuracy went from 90.5 to 93 which is a good jump. The problem is though that this improvement is not consistent, I ran this code several times while trying to write the report, and most of the time I was getting almost identical results. Even one time the first model outperformed the second model by 1%, so I would say it worths it most of the time, but not always.

Q7: Error Analysis

After training and evaluating multiple models, it's important to move beyond the overall accuracy and dive deeper into what kinds of mistakes the model actually makes.

In this task, we write a function that allows us to look at individual test examples, grouped into:

- True Positives (Spam correctly predicted)
- True Negatives (Normal correctly predicted)
- False Positives (Normal predicted as Spam)
- False Negatives (Spam predicted as Normal)

We call the function `analyze_predictions_cleaned()` and pass in:

- The trained model
- The original test DataFrame (so we can display the actual text)
- The test DataLoader
- The device (cpu or cuda)

The function does the following:

1. Collects all the predicted labels and probabilities from the model.
2. Pairs them with the actual labels and email texts.
3. Groups the results into TP, TN, FP, and FN categories.
4. Displays up to “num_examples_to_show” from each group, including the actual text (first 500 characters), label, prediction, and probability.

We also clean up the output — instead of printing long stat summaries or confusing tables, we just show the actual examples. That's usually much more informative.

Analyzing the results

True Positive

These usually include:

- Marketing tone (e.g., "هدیه بگیرید", "پیشنهاد ویژه")
- Lots of product mentions, brand names, and prices
- External URLs and referral links
- Repeated use of formatting: ***bold***, special characters, or excessive punctuation

True Negatives

Most of them involved:

- Personal questions, discussions, or academic conversations
- News-style text, political commentary, or technical content
- Clean language, no URLs, and natural sentence structure

False Positive

After reading them, here are a few hypotheses:

1. **Formal complaint tone:** These emails contain formal wording and structure (e.g., a letter to the ministry, customer complaints, etc.), which may resemble spam-style formatting, especially if they mention names, addresses, or dates. Like:

گزارش نصب لباسشویی، نامه به آموزش و پرورش، شکایت درباره بلیت هواپیمایی...

2. **Use of strong keywords:** Even if the email is not spam, the presence of certain keywords (like “محصولات”، “درخواست”، “مشکل”، “قیمت”) might trigger spam detection if they appear too frequently or resemble promotional templates.

3. **Misleading structure:** Emails with blocks of bold phrases, lists, or highlighted text might resemble spam HTML formatting.

These are understandable mistakes. The model seems to be slightly over-sensitive to formal or templated emails.

False Negative

These examples are often trickier to spot:

1. **Disguised spam:** Some spam messages open politely or use “neutral” language before linking to a spam product. Like:

“سلام دوست عزیز – لیست محصولات با قیمت پایین رو ببینید ...”

2. **Medical or educational content:** One false negative talked about نازایی در زنان — it may have looked like health content, but it had the structure of a spam post.

3. **Low confidence spam:** Some spam had lower confidence (e.g., 0.28 probability), suggesting the model was unsure. This means the language might’ve been close to ham in tone or structure, even though the content was promotional.

The model could benefit from training on more subtle spam or adding linguistic features to help catch these edge cases.